9 Discrete and Sequencing Control

By Bob Kretschmann and Jim Christensen

Topic Highlights

Discrete/Sequential Control Concepts and Hardware Systems
Basic Functional Structure of a Programmable Controller System
User's Control Objectives and Application Requirements
Selecting a PLC System
Software, Programs, and Programming Languages

9.1 Introduction

Discrete control is concerned with finite numbers of stable states—devices such as on/off valves, pumps and manifolds. The interest of the user is in an orderly transition from one state to the other, and whether a situation is normal or abnormal.

A related aspect of control in common industrial processes is known as sequential control. With sequential control, a process moves through a succession of distinct states, usually carried out by discrete control functions, but also occasionally by manipulating process outputs and monitoring process inputs.

To introduce this section for new engineers reading this, here are basic definitions, taken from ISA's *Automation, Systems, and Instrumentation Dictionary* (2003):

- **Discrete control** – On/Off control. One of the two output values is equal to zero.
- **Sequential control** – A class of industrial process control functions in which the objective of the control systems is to sequence the process units through a series of discrete states (as distinct from continuous control).

9.2 Discrete/Sequential Control Concepts and Hardware Systems

One of the principal means used by industry to achieve or execute discrete and sequential control methodologies is the programmable (logic) controller (PLC).

This technology has become so important that the International Electrotechnical Committee (IEC) created and maintains the standard IEC 61131, comprised of seven parts that describe, define, and provide application guidance for PLCs.

In this standard, a PLC is defined as:

> *Digitally operating electronic system, designed for use in an industrial environment, which uses a programmable memory for the internal storage of user-oriented instructions for implementing specific functions such*

as logic, sequencing, timing, counting and arithmetic, to control, through digital or analog inputs and outputs, various types of machines or processes. Both the PLC and its associated peripherals are designed so that they can be easily integrated into an industrial control system and easily used in all their intended functions.

Here is a basic definition, taken from ISA's *Automation, Systems, and Instrumentation Dictionary* (2003):

PLC – Programmable Logic Controller – *1. A controller, usually with multiple inputs and outputs, that contains an alterable program (ANSI/ISA-5.1-1984[R1992]).*

9.2.1 Programmable Logic Controllers

Programmable logic controllers (PLCs) are high-speed control computers. Typical features of a PLC are:

- a high-speed CPU
- flexible I/O systems with specialized cards for handling motion control
- a separate human machine interface

PLCs were traditionally programmed using relay ladder logic. Ladder logic is a powerful language for handling electrical motor control, but though it replaces traditional hardwired control, it can be considered a low-level programming language. Adding more advanced elements to traditional ladder logic can give the programmer the power to design complex control applications.

The PLC offers a very flexible programming environment. However, it still does not offer all the built-in functionality of a distributed control system (DCS) in terms of sharing process information through tags and tracking data quality. PLCs offer redundancy features for most critical components (power supply, CPU, I/O). However, exploiting these features typically requires more effort from programmers than a DCS requires. PLCs are now used for medium- and large-size control of analog process variables.

9.3 Basic Functional Structure of a Programmable Controller System

The general structure with main functional components in a programmable controller system is illustrated in the Figures 9-1, 9-2 and 9-3. These functions communicate with each other and with the signals of the machine/process to be controlled.

The operation of most PLCs consists of a repeated cycle of four major steps.

1. All inputs from appropriate interfaces are scanned, providing a consistent "image" of control input data. These represent field sensors such as switches, proximities, temperatures, data from other PLCs, etc.
2. A single "scan" is made of the user's control program, calculating or deriving a new "image" of control output data. At the same time all program variables, timers, counters, etc., are updated.
3. The new "image" of control output data is transferred to the appropriate interfaces for transfer to the target control devices. These represent field actuators such as relays, motor drive commands, displays, data to other PLCs, etc.
4. Finally, "housekeeping" tasks are performed on a time available basis. These may include communication with operator stations, some supervisory communication, diagnostics routines, etc.

After completion of the "housekeeping" the cycle repeats.

There are variations on this method, but, in general, this represents the overall process.

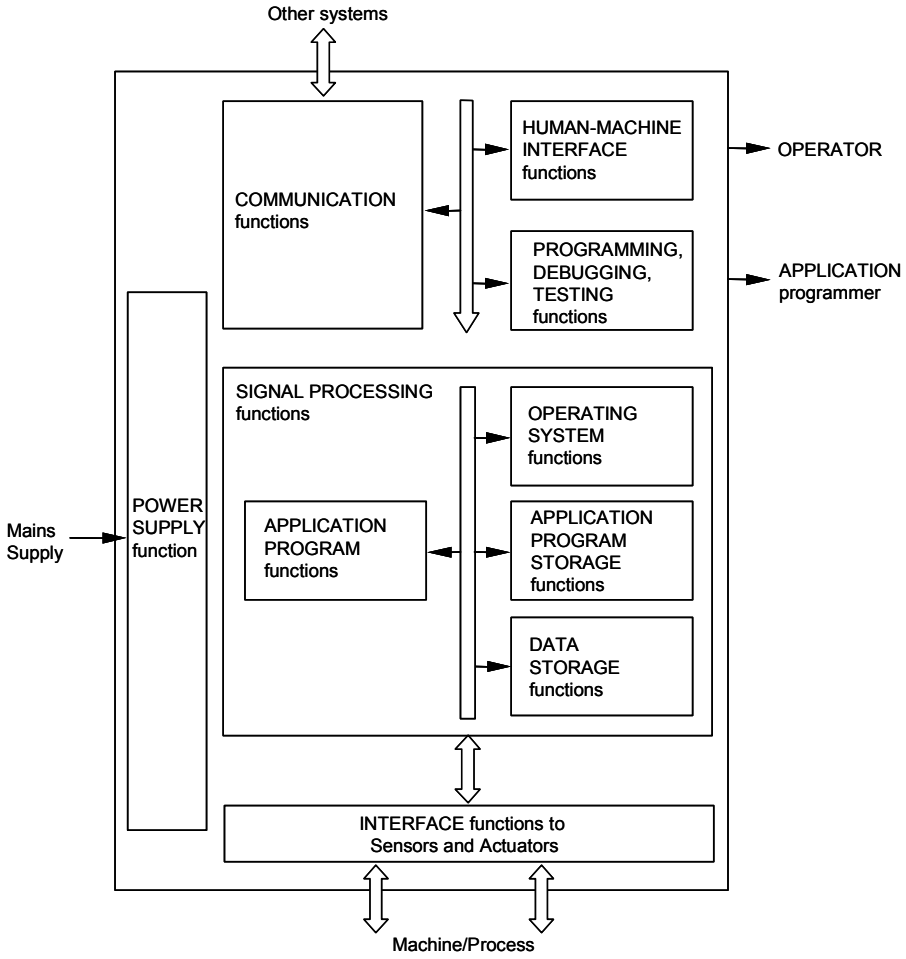


Figure 9-1: Basic Functional Structure of a PLC System

Some programmable controller systems with separate I/O and/or communications processors provide for overlapping the scanning of the user program with the scanning of the inputs (Step 1) and outputs (Step 3) and communication functions (Step 4). In these cases, special programming mechanisms may be needed to achieve concurrency and synchronization between the program and I/O scans, and between the program and communications processing.

A further feature of some PLCs is the incorporation of a multi-tasking operating system. As in general computing, the PLC operating system serves to coordinate the multitude of hardware and software resources and capabilities of the PLC. The incorporation of a multi-tasking operating system serves to allow the PLC to essentially execute multiple instances of the basic operation cycle described above, rather than a single instance, at the same time. This affords much increased flexibility and capability over a single-tasking operating system PLC.

In PLCs incorporating multi-tasking operating systems, mechanisms may be needed to achieve appropriate levels of task coordination and synchronization between the multiple instances of the basic operating cycle. As an example, one task's results may affect another task's decisions—especially with regard to I/O.

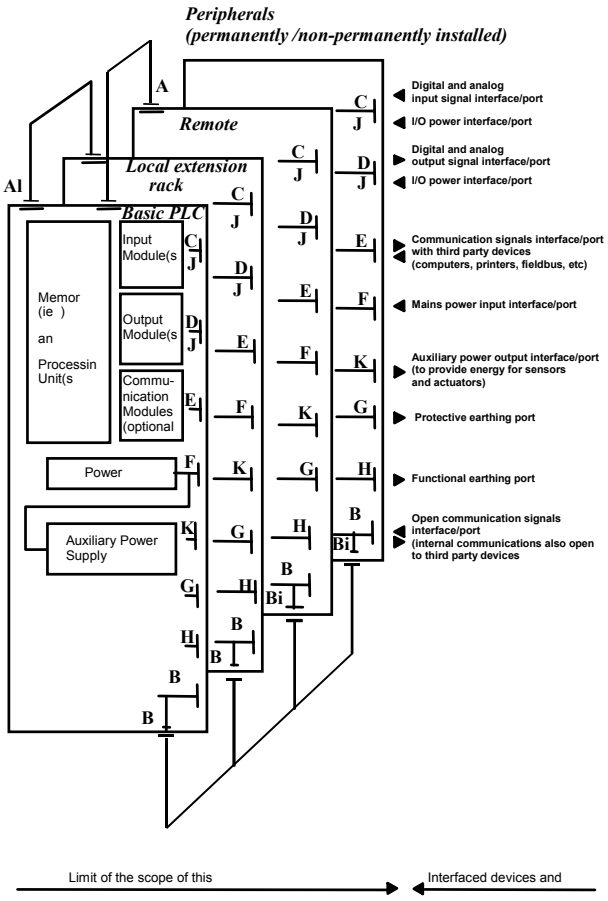


Figure 9-2: Programmable Controller Hardware Model

Al	Communication interface/port for local I/O
Ar	Communication interface/port for remote I/O station
Be	Open communication interface/port also open to third party devices (e.g., personal computer used for programming instead of a PADT)
Bi	Internal communication interface/port for peripherals
C	Interface/port for digital and analog input signals
D	Interface/port for digital and analog output signals
E	Serial or parallel communication interfaces/ports for data communication with third party devices
F	Mains power interface/port. Devices with F ports have requirements on keeping downstream devices intelligent during power up, power down and power interruptions.
G	Port for protective earthing
H	Port for functional earthing
J	I/O power interface/port used to power sensors and actuators
K	Auxiliary power output interface/port

Figure 9-3: Interface/Port Descriptions for Figure 9-2

9.4 User's Control Objectives and Application Requirements

With clear objectives and application requirements in mind, discussions with the vendor about controllers become immeasurably easier. This design information is essential and should include a general description of the process or equipment to be controlled. This will enable the user and the vendor or manufacturer to establish general equipment requirements and considerations.

Major topics that should be considered:

- process to be controlled
- user control objectives
- general control and operation requirements
- plant and personnel protection considerations
- general installation and environmental considerations
- expansion and integration requirements
- hardware configuration and regulations
- system availability requirements
- equipment breakdown implications
- spare parts requirements
- redundancy
- applicable local/national/international standards and regulations
- performance requirements
- interface to other systems
- maintenance
- cable wiring, routing and termination
- company regulations
- documentation: content, format
- regulatory/certification/approval requirements
- delivery and equipment installation schedule
- engineering responsibility: hardware, software, documentation, protective measures, testing, commissioning
- other considerations, as required

9.4.1 User System Description

This subclause refers to information to be presented to the vendor by the user as shown by the dashed lines in Figure 9-4.

The user must provide the vendor information about the existing system engineering, including current information about any third-party engineer. The vendor should be apprised of plant engineering, as well—both production engineering and maintenance engineering.

This system description should succinctly define the user's objective and give the vendor clear and relevant information concerning operations, monitoring, and configuration of any hardware and software. Additionally, corresponding documentation, such as diagrams, drawings, and signal (I/O) sequence and timing requirements should be used as support.

9.4.2 User System Characteristics

User system characteristics include such matters as:

- continuous or batch processing
- loop (PID) control
- distributed control
- changing of process (recipe supporting)
- downloading

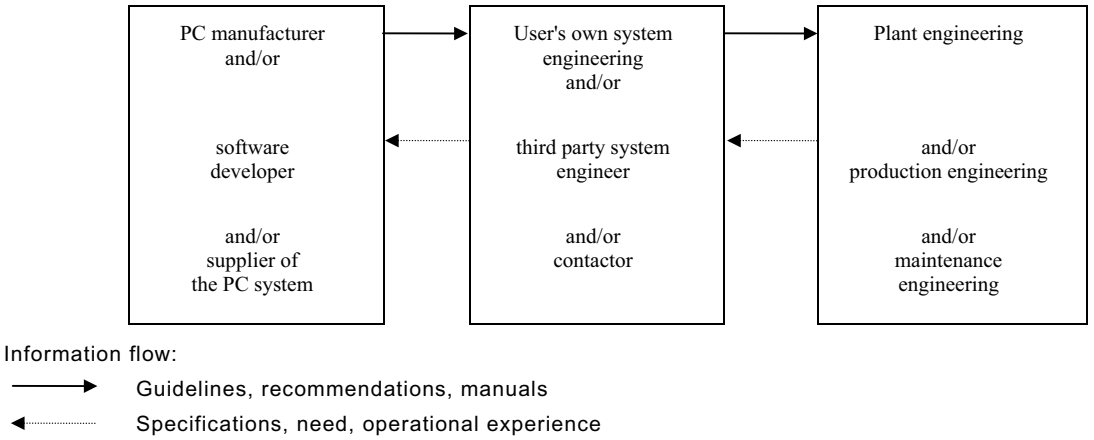


Figure 9-4 : System Description Information Flow

- autonomy of local stations
- system availability requirements
- total system response time
- redundancy
- multitasking
- alarm handling
- trending
- operator interface
- operator prompt
- remote supervision
- manual over-ride
- protection/safety considerations
- interlocks
- dynamic characteristics
- non-linearities
- authorization
- normal shutdown
- automatic restart
- data communication
- peripherals
- networks (LAN, WAN)

9.4.3 Control System Parameters

Important elements of the control system should be defined when process control is required. This can include the specific operation of the plant and equipment, the specific allowable process variables and values, the limits for actions and reactions, etc.

Other topics, that should also be considered, include:

- I/O listing
- sensors: types, signal level, power requirements
- signal conditioning
- control outputs: types, power requirements
- data transfer/access
- local/remote display

- logging and archiving
- electrical interference rejection criteria
- system availability
- I/O redundancy: single or voting (e.g. two out of three)

9.4.4. Alarms

The user's system description should include alarm requirements. The organization concept, priority, alarm operation and display should be defined.

Other major topics to be considered include:

- alarm sensing methods
- first fault identification
- fault discrimination
- dedicated alarm display
- alarm at man-machine interface
- alarm acknowledgment
- alarm logging

9.4.5 Human-Machine Interface

Human-machine interface (HMI) requirements include considerations for operator intervention, access control, and security arrangements.

Other major topics include:

- multistation display
- dedicated display areas
- dynamic graphics
- access control, authorization and passwords
- key locks
- software locks
- keyboard, track ball, mouse, touch screen, etc.
- ergonomics

9.4.6 Interlocks, Sequencing

Interlocks are used to arrange the control of machines or devices so their operation is interdependent, in order to assure they are coordinated properly (ISA-RP55.1-1975 [R1983]).

Interlocks may be:

- A physical device, equipment, or software routine that prevents an operation from beginning or changing function until some condition or set of conditions is fulfilled;

- A device, such as a switch, that prevents a piece of equipment from operating when a hazard exits;

- A device used to prove the physical state of a required condition and to furnish that proof to the primary control circuit;

- A device or group of devices (hardware or software) that are arranged so to sense a limit or off-limit condition or an improper sequence of events. To avoid an undesirable condition, they then shut down the offending or related piece of equipment or prevent it from proceeding in an improper sequence. (ANSI/ISA-77.44.01 &.02-1995).

In a similar fashion, sequencing is the ordering of overall process events, such that one control process properly completes its assigned task before handing off control to the next task process in the overall event path.

Major considerations for the interlocks and sequencing include:

- physical requirements of interlock
- types of systems to be interlocked
- systems and system communication
- requirements for data network.

9.4.7 System Outage

Topics relating to system outage include the following:

- power supply configuration
- system back-up
- diagnostics
- failure mode
- failure display levels: system, module, or card
- restart: cold, hot, or warm
- protection of personnel and equipment.

9.5 Selecting a PLC System

9.5.1 Main processing unit

The user should refer to IEC 61131-1 and IEC 61131-2 for relevant details, as well as Table 9-1 for a listing of selection considerations.

9.5.2 Human-Machine Interface

The human-machine interface (HMI) should be carefully specified and selected because it will be the operator's window to the plant control system. It may also be the access facility for programming and fault diagnosis for the PC system. Selection criteria for HMI are listed in Table 9-2.

Table 9-1: Main Processing Unit—Selection Criteria

Criteria	Comments and considerations
User program memory	The organization and size of user application program memory
Memory back-up	Power back-up for volatile memory
System hardware configuration	- Racks - Cables - Bus expanders - Power supplies - Number of I/O modules per type - Memory allocation per I/O type - etc.
Programming languages supported	- Languages supported by the MPU - Conformance to the P.C. language standard - Any differences in objects, instructions, semantic and syntactic rules should be noted
Scan time	The calculation of scan time which includes: - Scan - Memory utilization - Transfer - Program execution - User's program diagnostics
I/O memory processing	i.e., use of I/O image registers periodically refreshed, “get/put” type instructions, interrupt and event-driven programs, etc. and their effects on system response times, including restart (cold, warm, hot restart)

Table 9-2: Human-Machine Interface—Selection Criteria

Criteria	Comments and considerations
Types	1) Integrated in PLC - uses PLC-MPU and memory, may restrict display features and PC operating features 2) Intelligent - permits extensive displays and operating features, which may include operator support
Display	- Brightness - Contrast - Screen size - Definition - Resolution - Color purity - Number of characters - Labels - Touch screen - Mouse - Rollerball - Display refresh time - Formats - Windows - Menus - Native language support
Keyboard	- Tactile feel - Ergonomics - Mouse, rollerball
Access control	- Operator and/or programmer - Keylock or software protected levels
Alarms	- Separate display - Screen windows - Active data points - Alarm management

9.6 Software, Programs and Programming Languages

Software is a general term that designates a set of computer programs designed to carry out specific tasks.

A program is a series of actions that a computer takes to achieve a certain result. Activities such as "locate," "read," "interpret," and "execute the instruction" are repeated in sequence for each instruction throughout the program.

A programming language is a set of representations, conventions, and rules used to convey information and instructions to the computer. The language of the digital electronic circuits inside the computer is comprised of the binary codes that represent the numbers, letters, symbols, and commands used by humans to give instructions to the computer. The language the human user uses must be converted into digital codes that the machine understands. This digital code is called machine language.

To facilitate communication between humans and machines, a high-level language is required. This is a general-purpose programming language such as C/C++, Java, FORTRAN, BASIC, Pascal, and ADA. These high-level languages are organized in a way that is directly related to the way humans solve problems. An intermediate language, called Assembly language, is also used to bridge the high level language with the machine language in the program compiler, although nowadays this is transparent to the user.

Many years ago, a programmer might have used assembly language to write a program using instruction abbreviations called mnemonics. The program was then converted into a machine-language program using an Assembler. When a facility purchases the digital control system, the computer is usually

preprogrammed to communicate in a high-level language. Some systems are also equipped with configurable prewritten programs for specific applications. These systems are referred to as a configurable digital system. They can be easily configured by engineers without specialized computer backgrounds. The global capability of these configurable digital systems includes all conventional control system functions plus many capabilities not easily implemented using analog hardware, including feed-forward, dead-time compensation, and multi-variable control.

Traditionally, control systems have been implemented using a wide variety of languages from Assembler to general-purpose language such as BASIC, FORTRAN, or C. Over the intervening years, in an effort to simplify and enhance the control engineers experience and effectiveness, programming languages specific to the needs of the industrial process control application space have been developed.

These languages are:

- *Relay ladder logic*. This programming language was initially a software rendition of the hardwired logic control that was implemented using electromechanical relays. The roots of this language lie in discrete machine control. Ladder logic has evolved extensively from its origins to incorporate many sophisticated control elements, including mathematical expressions (integer and real), controllers (PID), and Table manipulation. The relay ladder logic language is traditionally used by programmable logic controllers (PLC).
- *Function block*. This language is a software representation of basic analog process control elements. Function block components include: scaling, alarming, PID, and many others. The function block language is used in single-loop controllers and distributed control systems (DCS).

The increasing demands of complete plant integration and the high level of automation now required by industrial operations have revealed many deficiencies in the conventional relay ladder and function block languages. Those shortcomings can be summarized as follows:

- Language implementations vary between different systems.
- It is difficult to reuse standard software elements (no object orientation).
- Complex data addressing using real address instead of symbolic variable names.
- It is difficult to program sequence operation.

9.6.1 IEC 61131 Features and Programming Languages

Because of the difficulties traditional programming languages pose for process control, the International Electrotechnical Commission (IEC) commissioned a standard to help standardize control system programming: IEC 61131, Part 3 (usually referred to as IEC 1131). It covers all important aspects of control systems programming.

The standard allows facilities to implement modern software architectures, for example: structured functional blocks, the definition of reusable software elements, and symbolic variable names.

This part of IEC 61131 specifies the syntax and semantics of a unified suite of programming languages for PLCs. These consist of two textual languages,

- Instruction List (IL), and
- Structured Text (ST)

and two graphical languages,

- Ladder Diagram (LD)
- Function Block Diagram (FBD)

Sequential Function Chart (SFC) elements are defined for structuring the internal organization of programmable controller programs and function blocks. Also, configuration elements are defined which support the installation of programmable controller programs into programmable controller systems.

The selection of software and hardware are complementary procedures and should be concurrently evaluated during system selection procedures. These PLC programming languages, defined within IEC 61131-3, are generally available for PLC hardware.

9.6.2 Sequential Function Charts (SFCs)

Sequential function chart (SFC) is a graphical language for depicting the sequential behavior of a control system (see Figure 9-5). It is used to define control sequences that are time and event-driven. SFC is an extremely effective graphical language for expressing both the high-level sequential parts of a control program and for programming low-level sequences, for example, to program an interface to a device.

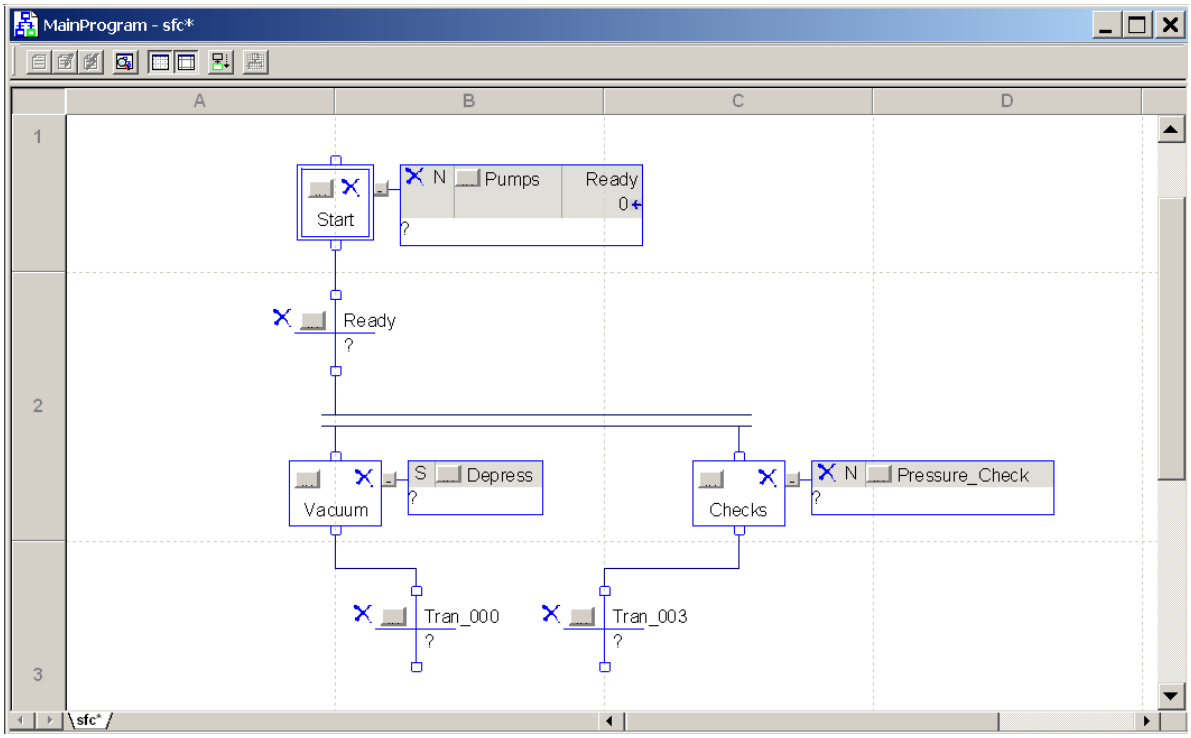


Figure 9-5: Sequential Function Chart

The SFC elements are used to structure the internal organization to perform sequential control functions in order to partition a set of steps and transitions that are connected by directed links. Only function blocks and programs can be structured using these.

A transition is the condition where the control passes from one step to another, along the corresponding directed link. Each transition step has an associated transition condition which is the result of evaluation by a single Boolean expression. Associated with each step will be zero or more actions.

For further information and additional details on best target applications (user guidance), as well as key features/capabilities, see reference IEC 61131-3, 2.6.

9.6.3 Instruction List

Instruction list (IL) is a low-level "assembler-like" language that is based on similar instruction list languages found in a wide range of today's PLCs (see Figure 9-6). An Instruction List program is composed of a series of relatively simple instructions. Precise syntax is required, although blank lines may be inserted between instructions. Each line must begin on a new line and contain an operator, along with optional modifiers, and, if necessary to the operation, one or more operands that must be separated by commas.

```
      LD    T1
    JMPC    Reset
      LD    Temp_1
      ST    Max_Temp
Set:  LD    0
      ST    D_V76
```

Figure 9-6: Instruction List

For additional details on best target applications (use guidance), as well as key features/capabilities, see reference IEC 61131-3, 3.2.

9.6.4 Structured Text (ST)

Structured text is a high-level textual language that encourages structured programming. It has a language structure (syntax) that strongly resembles Pascal and supports a wide range of standard functions and operators. The IEC 61131-3 standard defines ST's formal syntax (see Figure 9-7). Structured text is based on traditional software programming language look and feel.

```
INT_CMD := MANUAL_CMD & MANUAL_MODE OR AUTO_CMD & NOT
AUTO_CMD_CHECK & NOT MANUAL_MODE

CMD_ON_TMR(IN := INT_CMD, PT := T_CMD_MIN)

ALRM_XY(S1 := CMD_ON_TMR.Q & NOT PERM, R := OK)

ALRM := ALRM_XY.Q1
```

Figure 9-7: Structured Text

For further information and additional details on best target applications (user guidance), as well as key features/capabilities, see reference IEC 61131-3, 3.3.

9.6.5 Ladder Diagram (LD)

Ladder diagram is a graphical language that is based on traditional relay ladder diagrams. Diagrams represent power flow. This language was the first commonly used language to program traditional

PLCs (see Figure 9-8). However, the IEC ladder diagram language also allows user-defined function blocks and functions to be interconnected so they can be used in a hierarchical design.

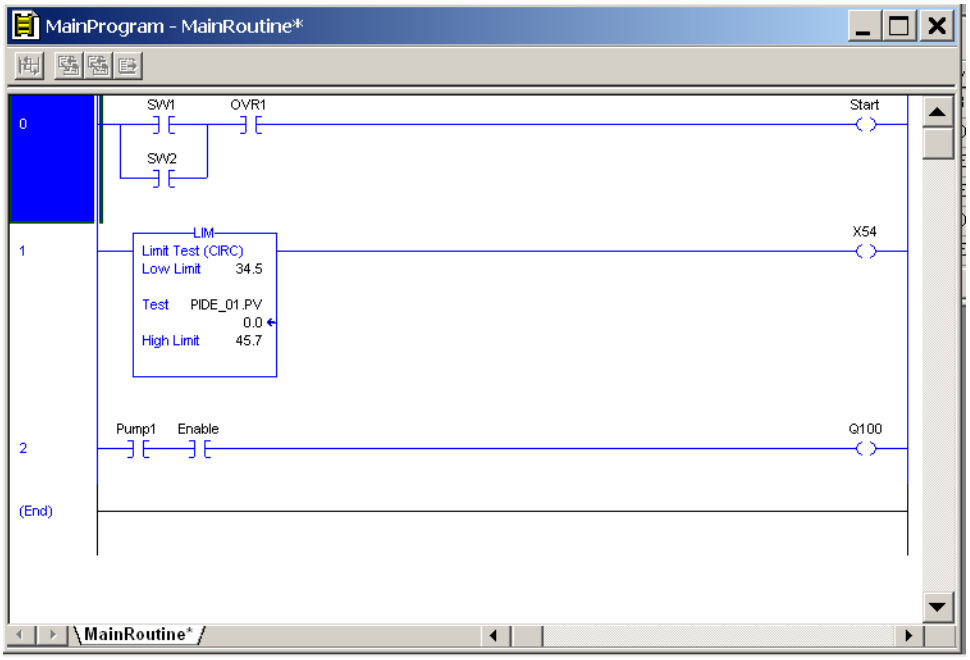


Figure 9-8: Ladder Diagram

For further information and additional details on best target applications (user guidance), as well as key features/capabilities, see reference IEC 61131-3, 4.2.

9.6.6 Function Block Diagram (FBD)

Function block diagram is a graphical language for depicting signal and data flows through function blocks that is, reusable software elements (see Figure 9-9). FBD is very useful for expressing the interconnection of control system algorithms and logic. It is based on the look and feel of the traditional logic diagram. In it, the diagrams represent data flow. The function block diagram is a network in which the nodes are function block instances, graphically represented functions (procedures), variables, literals, and labels.

For further information and additional details on best target applications (user guidance), as well as key features/capabilities, see reference IEC 61131-3, 4.3.1.

9.6.7 Compatibility of 61131 Languages

IEC 61131-3 takes into account the different courses of evolution of programmable controllers in North America, Europe, and Japan, and the wide variety of applications of programmable controllers in modern industry.

Figure 9-10, 9-11, and 9-12 below show the application of the LD, ST, and FBD languages to implement a simple command execution and monitoring function. In general, a desired functionality can be programmed in any one of the IEC languages. Hence, languages can be chosen depending on their suitability for each particular application. And, this demonstrates the mutual compatibility of the languages.

Programming discrete control devices for continuous control functionality is covered under an earlier topic in the book.

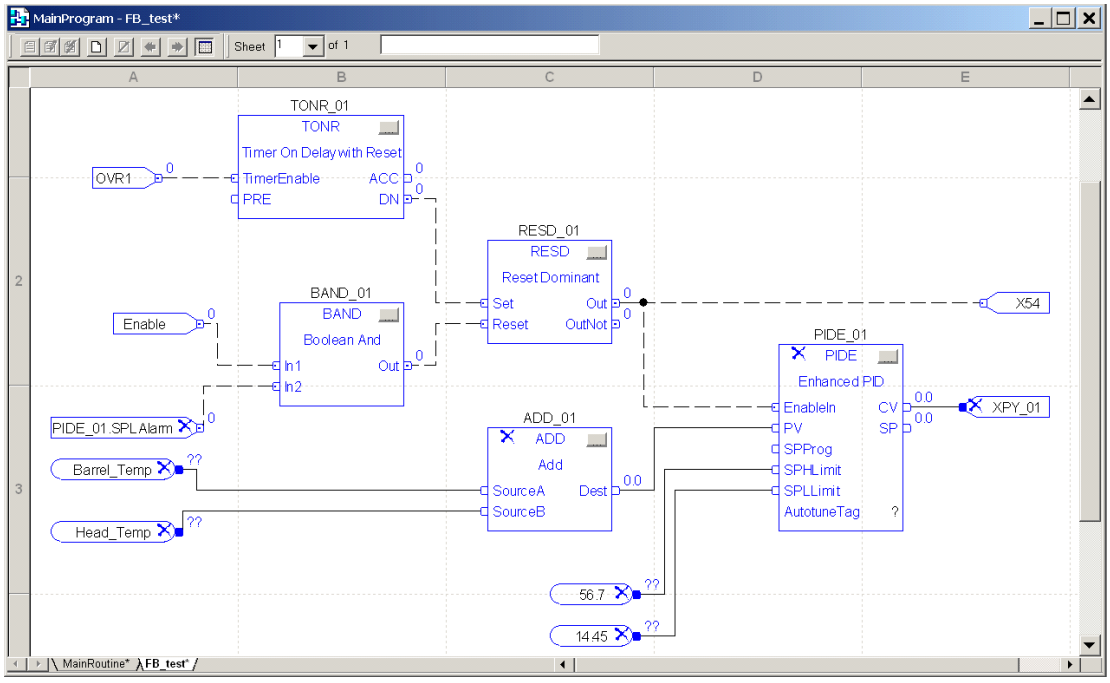


Figure 9-9: Function Block Diagram

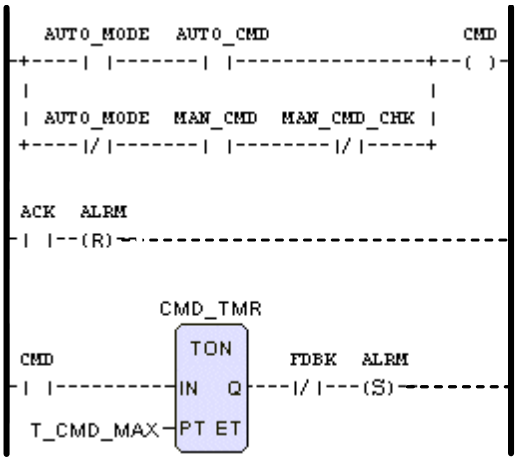


Figure 9-10: Application Using Ladder Diagram Language

```
CMD := AUTO_CMD & AUTO_MODE OR MAN_CMD & NOT MAN_CMD_CHECK & NOT
AUTO_MODE
CMD_TMR(IN := CMD, PT := T_CMD_MAX)
ALRM_FF(S1 := CMD_TMR.Q & NOT FBDK, R := ACK)
ALRM := ALRM_FF.Q1
```

Figure 9-11: Application Using Structured Text Language

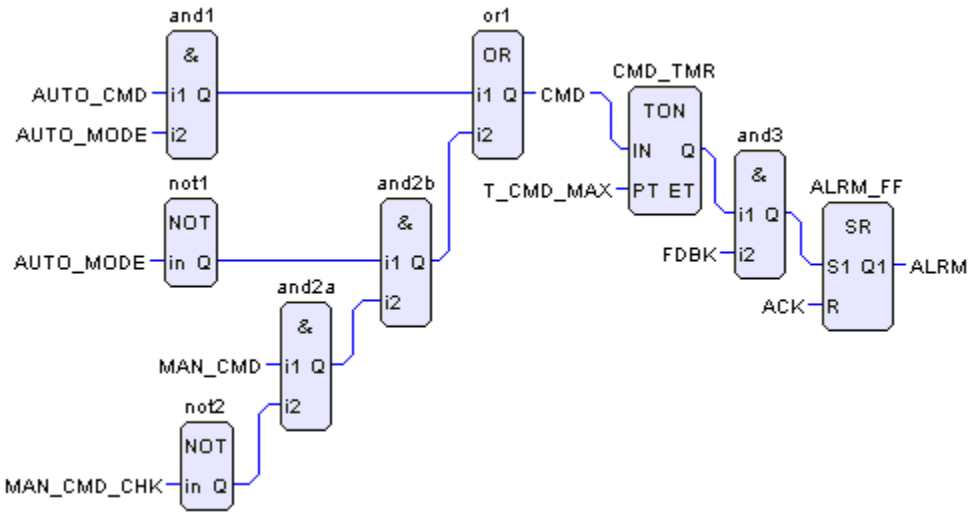


Figure 9-12: Application Using Function Block Diagram Language

9.6.8 System Software

Major topics to be considered include:

- programming structure/programming language
- PADT
- access to control programs: access methods, authorization
- documentation
- computer aided engineering tools
- on-line/off-line configuration.

9.7 References

The Automation, Systems, and Instrumentation Dictionary. Fourth Edition. ISA, 2003.

Coggan, D.A., Editor. *Fundamentals of Industrial Control.* Second Edition. Practical Guides for Measurement and Control Series. ISA, 2005.

Hughes, Thomas A. *Programmable Controllers*. Fourth Edition. ISA, 2004.

IEC 61131 – *Programmable Controllers*.

IEC 61131-1 Ed. 2.0 en:2003. *Part 1: General information.*
IEC 61131-2 Ed. 2.0 en:2003. *Part 2: Equipment requirements and tests*.
IEC 61131-3 Ed. 2.0 en:2003. *Part 3: Programming languages.*
IEC/TR 61131-4 Ed. 2.0 en:2004. *Part 4: User guidelines.*
IEC 61131-5 Ed. 1.0 en:2000. *Part 5: Communications.*
IEC 61131-7 Ed. 1.0 b:2000. *Part 7: Fuzzy control programming*.
IEC/TR 61131-8 Ed. 2.0 en:2003. *Part 8: Guidelines for the application and implementation of programming languages.*

John, Karl-Heinz and Michael Tiegelkamp. *IEC 61131-3: Programming Industrial Automation Systems – Concepts and Programming Languages, Requirements for Programming Systems, Aids to Decision-Making Tools*. Springer, 2001.

Lewis, R.W. *Programming Industrial Control Systems Using IEC 1131-3.* Revised Edition. IEEE, 1998.

About the Authors

Robert J. Kretschmann's 25-years at Rockwell Automation include industrial automation experience in product development, advanced research, and internal consulting. He has five additional years of experience in telecommunications and other work for the U.S. Air Force. Standards activities include: IEC TC65 Industrial-Process Measurement and Control Advisory Group, Technical Advisor; IEC USNC Technical Advisor for SC17B, Low Voltage Switchgear and Controlgear; IEC USNC Co-Technical Advisor for SC65B, Devices; NEMA THSC 17 Working Group – Control Circuit Device; CANENA THC 01IS; IEC Convenor TC65/SC65B/WG7 Programmable Logic Controllers; NEMA 1-IS Programmable Controllers Sub-Committee 22, Chairman; IEEE Standards Coordinating Committee 22, Power Quality – 1994-1997; and NEMA 1-IS Industrial Proximity & Presence Sensors Sub-Committee 15. A member of both ISA and IEEE, Kretschmann holds multiple U.S. patents. He frequently lectures, writes, and edits books and papers.

James Christensen retired from Rockwell Automation in April 2005 after a 23-year career. At Rockwell, he led development of the IEC 61131-3 Standard for Programming Languages for Programmable Controllers, which has become the global standard for this segment of the automation and control market. He also led development of the IEC 61499 standard, the successor to IEC 61131-3 for the next generation of industrial automation and control. His Function Block Development Kit (FBDK) software is the first, internationally used IEC 61499-compliant software tool kit. Before joining Rockwell, he worked two and one-half years with the Electronic Controls Division of Texas Instruments and three years with Strider Systems, Inc. He also was employed for eight years on the faculty of the University of Oklahoma, culminating in a tenured position as Associate Professor of Chemical Engineering; one year as a Ford Foundation Postdoctoral Fellow at Thayer School of Engineering of Dartmouth College; and two years as a research assistant at the Solar Energy Laboratory of the University of Wisconsin. Christensen is currently President of Holobloc, Inc., an Ohio-based corporation providing software tools, consulting and training for the application of IEC Standard 61499. His achievements in pioneering applications of object-oriented programming earned him the Rockwell International Engineer of the Year and Lynde Bradley Innovation Awards in 1991. He has written over 30 papers and book chapters, as well as delivered invited lectures and keynote addresses. He is a member of ISA and IEEE.